

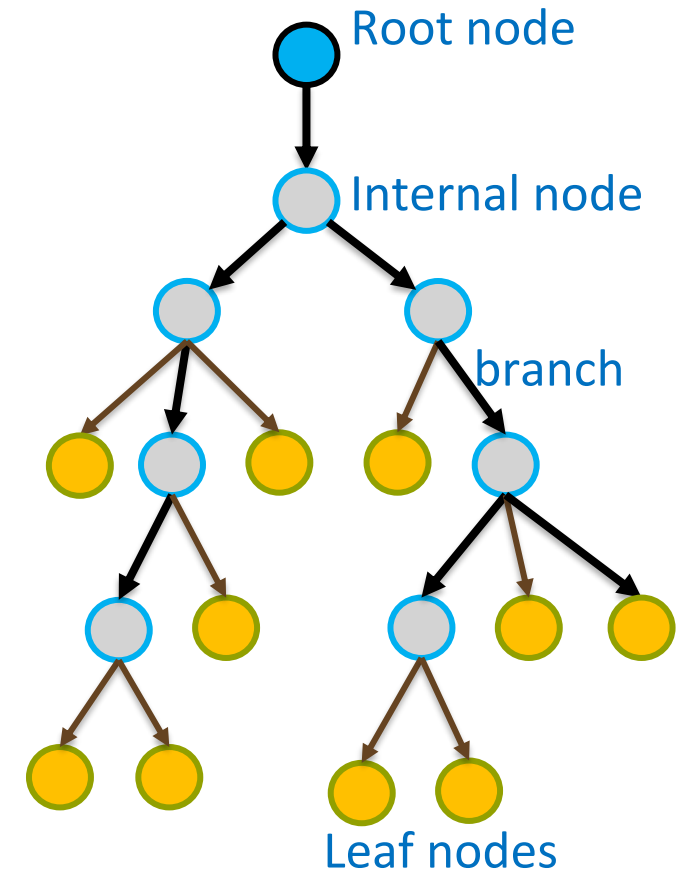
Decision Trees and their Variants

Contents

- Introduction to Decision Trees
- Decision Tree for Classification
- Decision Tree for Classification: Feature Space Perspective
- Building a Decision Tree for Classification
- Regression Trees
- Tree Bagging
- Tree Boosting
 - Gradient Boosting
 - XGBoost
 - LightGBM

Introduction to Decision Trees

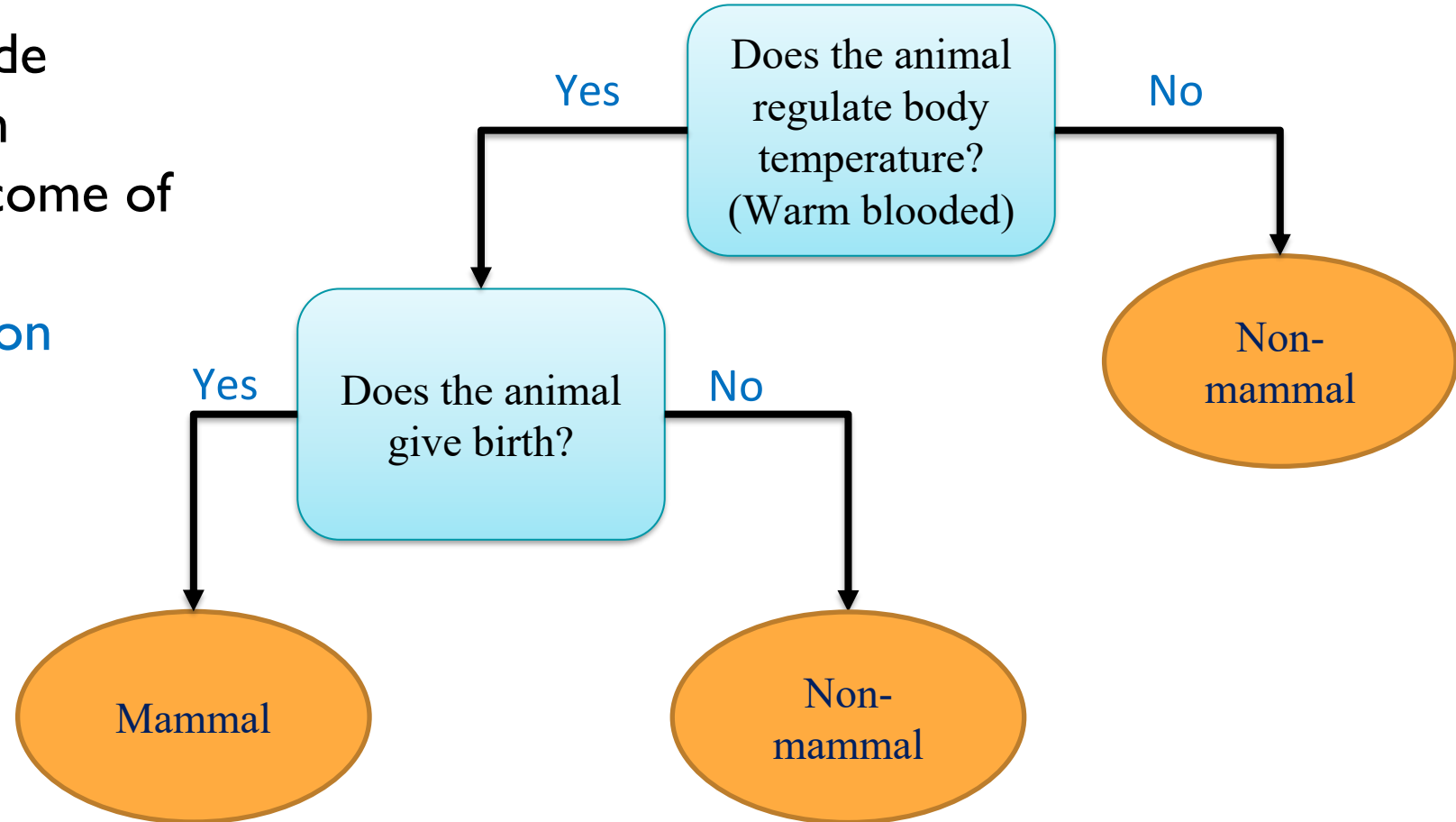
- One of the most popular tools in machine learning used for both classification and regression tasks
- Decision trees are also the basis for other machine learning techniques such as bagging, random forest, boosting, etc.
- **Key Idea:** A decision tree is a rooted tree that represents a set of prediction rules obtained by recursively partitioning the feature space



A rooted tree

Decision Tree – Intuition & Example

- **Intuition:** Each root/internal node represents a question/condition
- Each branch represents an outcome of the question
- Each leaf node denotes a **decision**

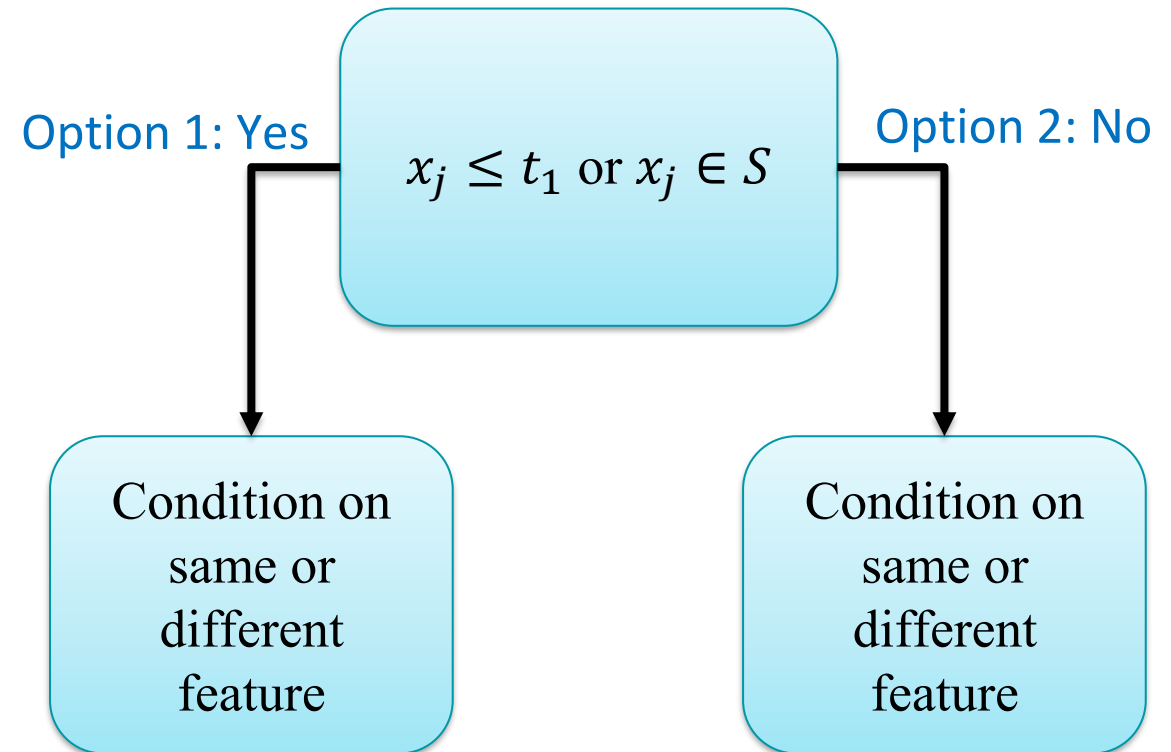


How to use a decision tree to classify data with multiple input features into different target classes?

A hand-designed decision tree to decide whether an animal is mammal or not

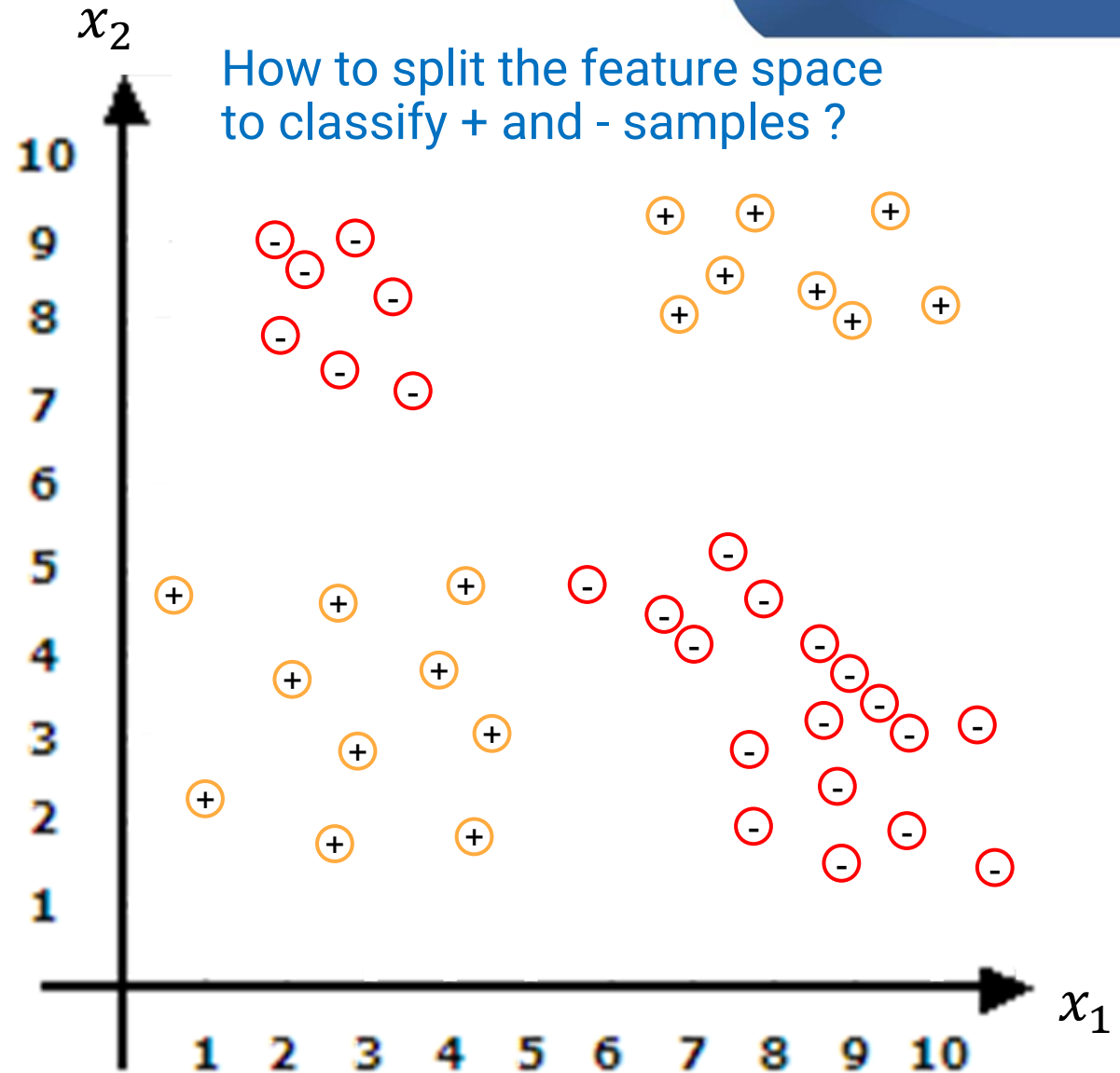
Decision Tree for Classification

- **Outputs:** Decision in this case is to decide the class to which sample belongs
 $y = \{1, 2, \dots, m\}$
- **Nodes:** Represent a question/condition on an input feature based on which tree splits into branches (conditional split)
- **Branches:** Represent different outcomes or options that are available based on the nodal condition
- Generally split is binary (yes/no) but multiway splits (discrete outcomes) are also possible

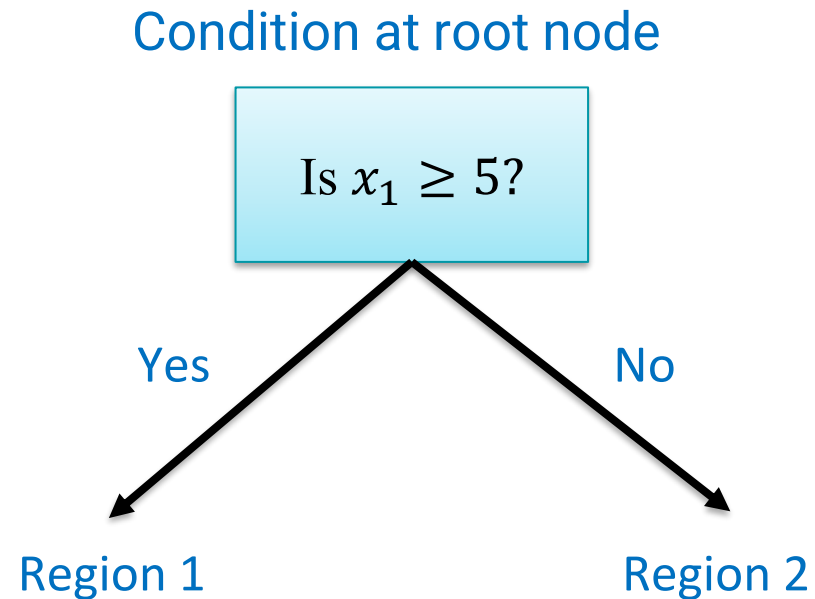


Decision Tree for Classification: Feature Space Perspective

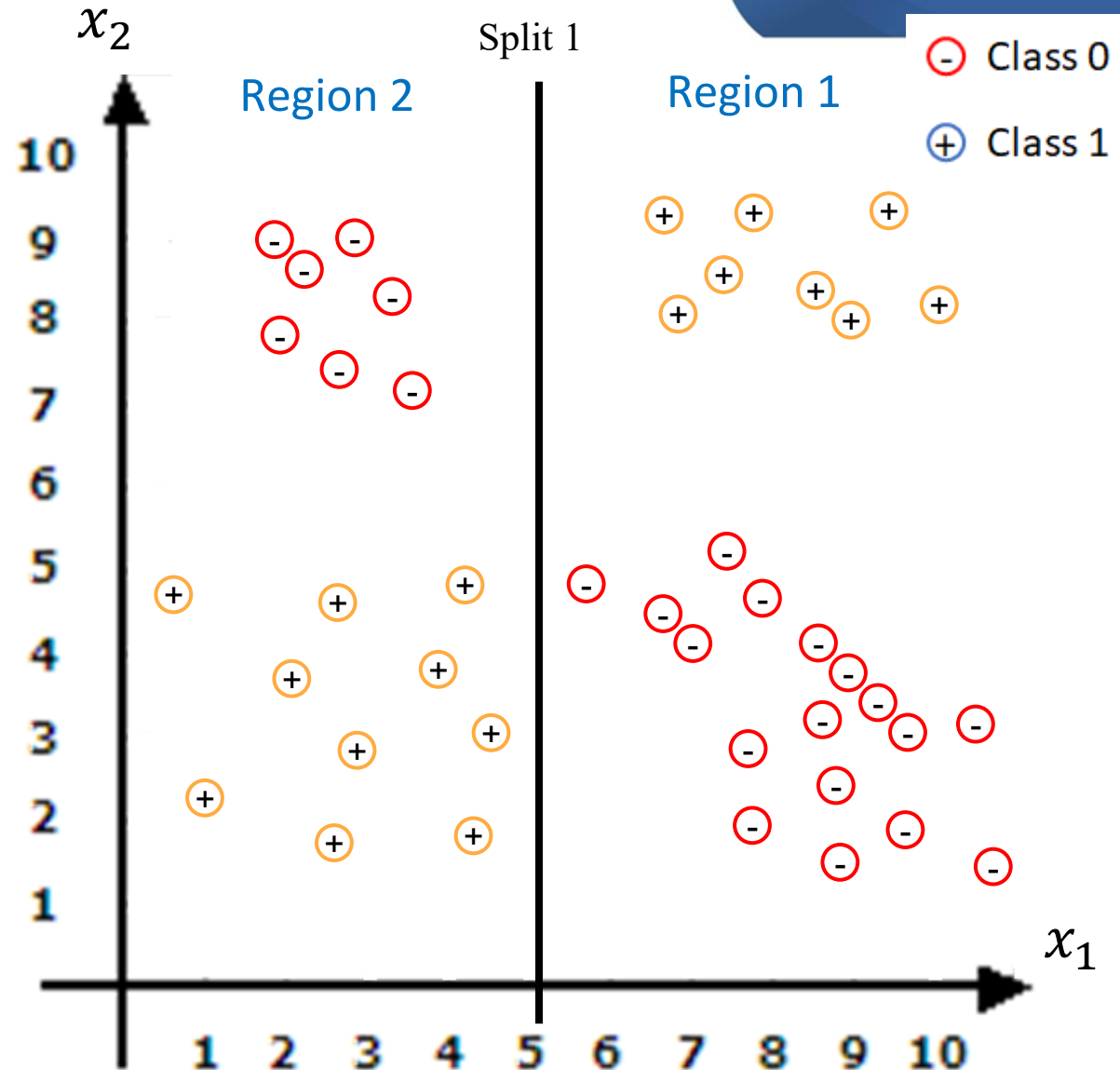
- **Input Space:** $\mathbb{R}^D$ or mixed numerical/categorical feature space
- Decision tree splits the feature space into multiple regions R_c based on the conditions on features
- Each region should contain samples belonging to one particular class or majority should belong to one class
- Then, each region is assigned a particular class c



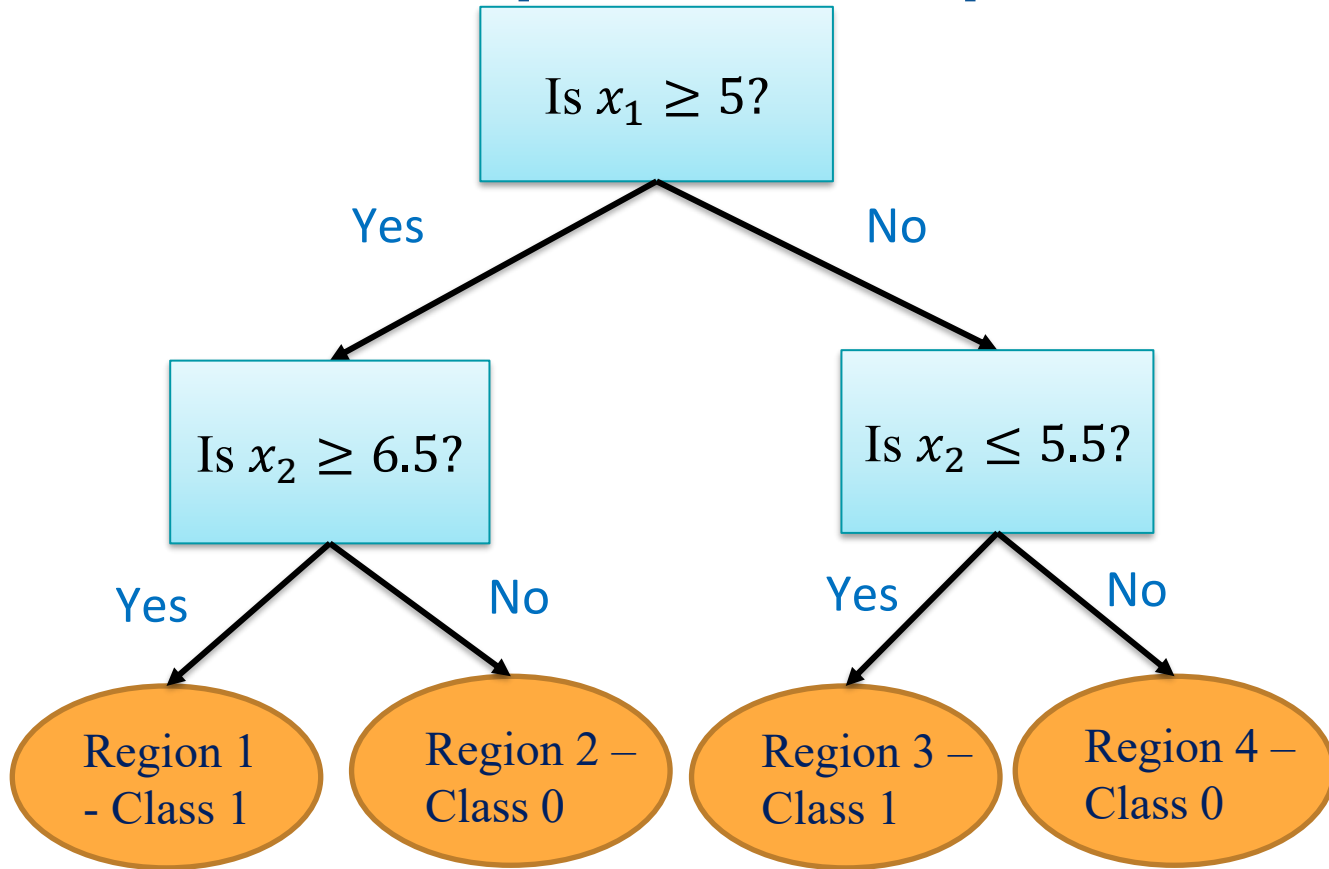
Decision Tree for Classification: Feature Space Perspective



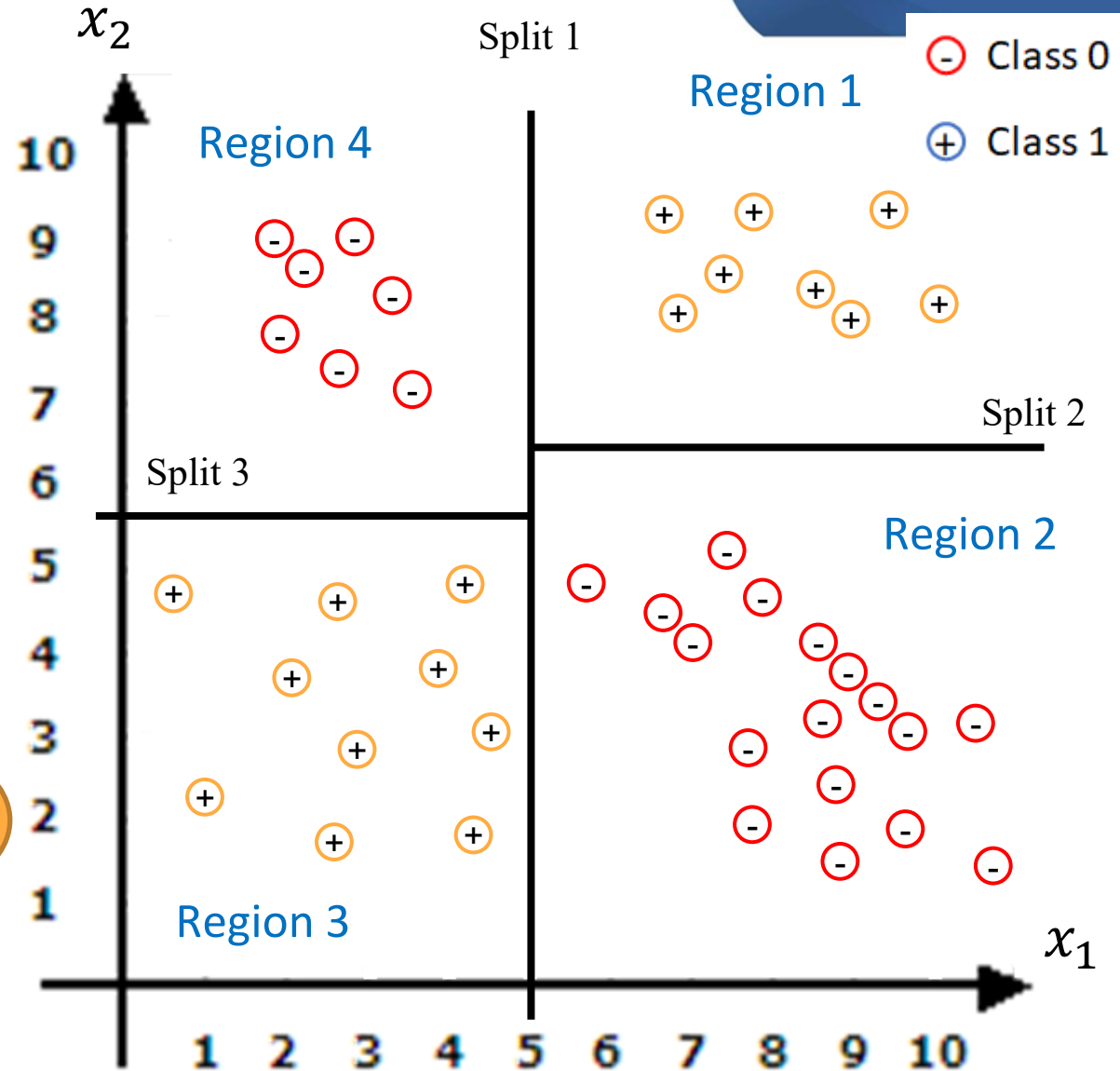
After split 1 at root node, the regions still contain samples of both classes



Decision Tree for Classification: Feature Space Perspective



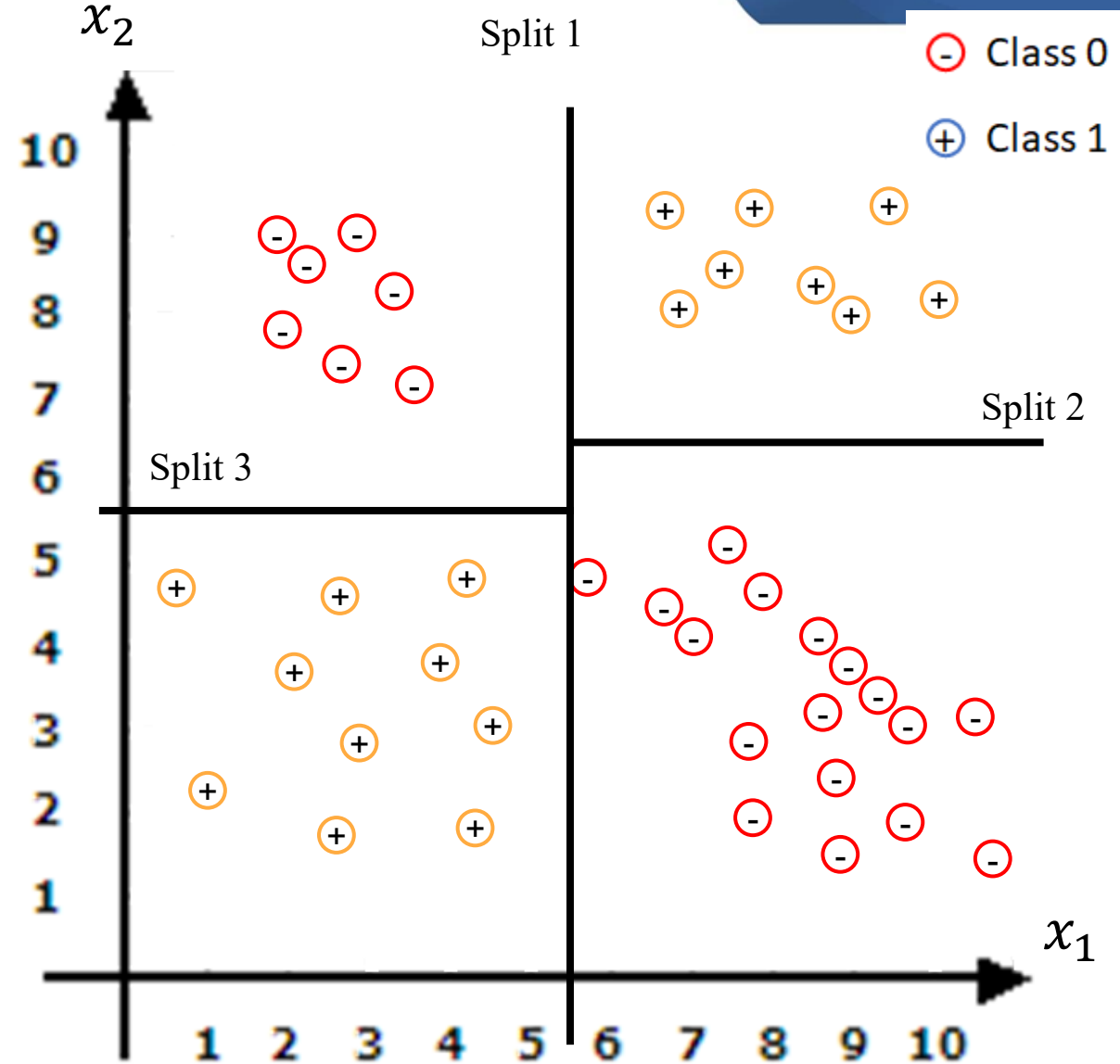
Each leaf node of the decision tree corresponds to a region in the feature space



Building a Decision Tree for Classification

Building a Decision Tree for Classification

- Questions to be answered while building a decision tree:
 - How to select the feature and the condition to split upon at a given node?
 - When to stop splitting or fixing a node to be the leaf node ?
What is the stopping criteria?

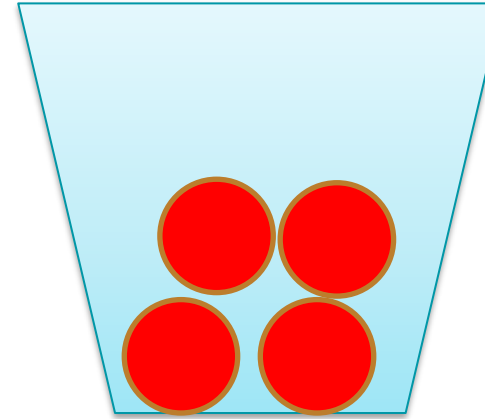


Building a Decision Tree for Classification: Feature and Condition to Split

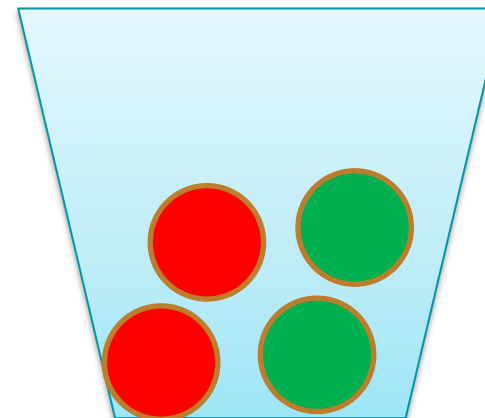
- Feature and condition for splitting can be selected randomly but for an efficient split, entropy or gini index are used to determine the split at a node

Gini Index and Entropy

- **Purity:** Indicates the homogeneity of a group of items (class of items)
- **Example:** Suppose there is a jar which can contain red and green balls
- How to quantify this purity mathematically when dealing with samples of different classes? – **Gini index and Entropy**



Jar 1 with highest purity



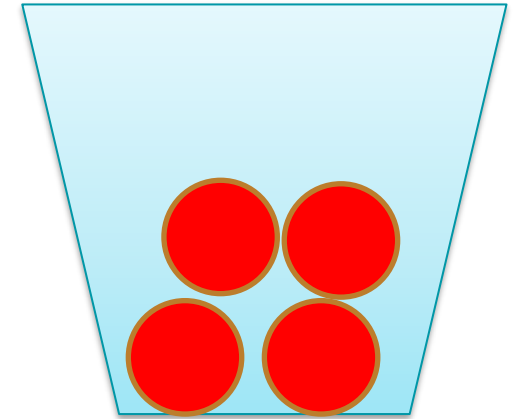
Jar 2 with least purity

Gini Index

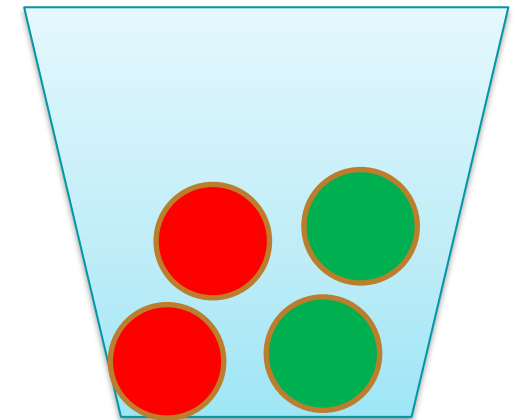
- **Gini index:** Measure of impurity of a set of samples

$$Gini = 1 - \sum_j p_j^2$$

- Jar 1: $Gini = 1 - (p_r^2 + p_g^2) = 1 - (1^2 + 0^2) = 0$
- Jar 2: $Gini = 1 - (p_r^2 + p_g^2) = 1 - (0.5^2 + 0.5^2) = 0.5$
- Gini index value of 0 indicates most pure set
- For two classes, the maximum Gini impurity is 0.5, attained when both classes are equally likely
- **Interpretation:** Probability of misclassification if a label is assigned at random according to the class proportions at that node



Jar 1



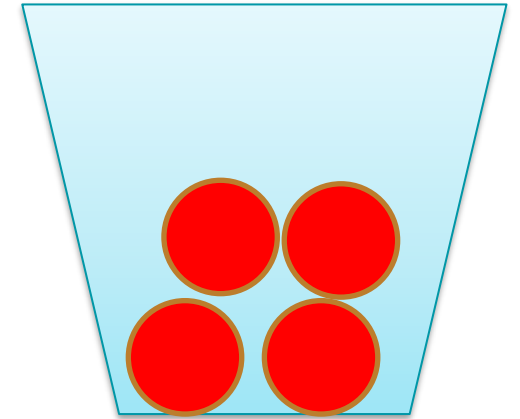
Jar 2

Entropy

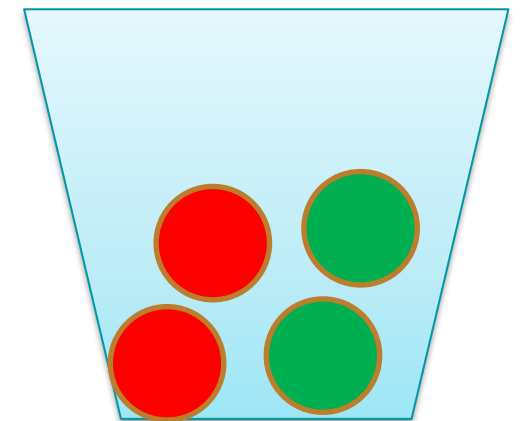
- **Entropy:** Measure of impurity of a set of samples
- **Definition:**

$$Entropy = - \sum_j p_j \log_2 p_j$$

- Jar 1: $Entropy = -(p_r \log p_r + p_g \log p_g) = (0 + 0) = 0$
- Jar 2: $Entropy = -(p_r \log p_r + p_g \log p_g) = -(-0.5 - 0.5) = 1$
- Entropy value of 0 indicates most pure set
- For two classes, the maximum entropy is 1, attained at $p_1 = p_2 = 0.5$



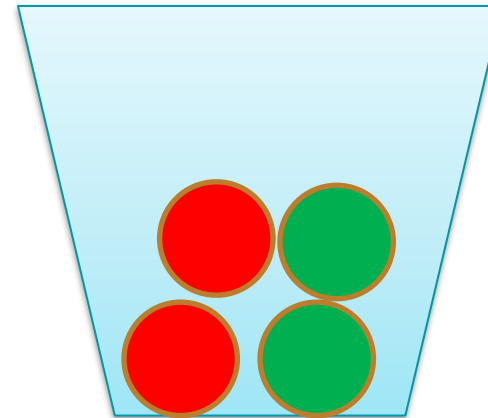
Jar 1



Jar 2

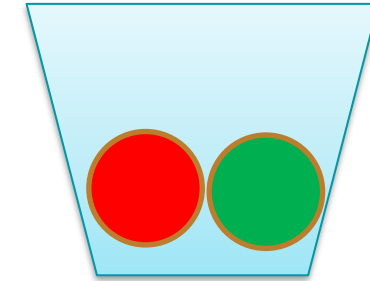
Building a Decision Tree for Classification: Feature and Condition to Split

- For an efficient split, entropy or gini index are used to determine the split at a node
- **Question:** How to use entropy or gini index to efficiently perform a split?
- **Answer:** Total Entropy or gini index should reduce after the split at a node



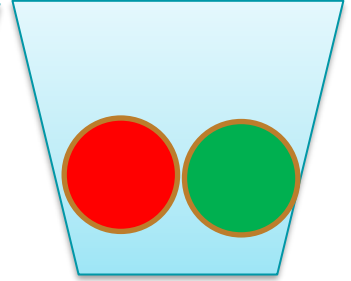
Jar 1

$Gini = 0.5$



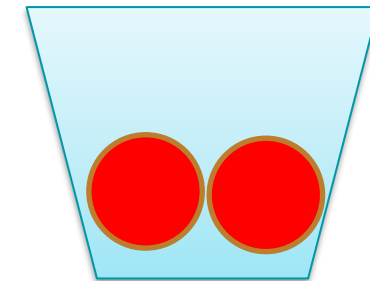
Jar 2a

$Gini = 0.5$



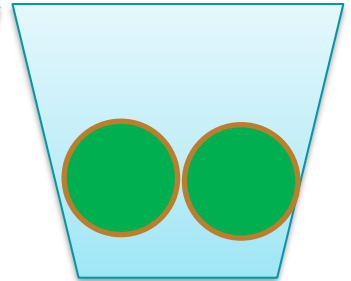
Jar 2b

$Gini = 0.5$



Jar 3a

$Gini = 0$



Jar 3b

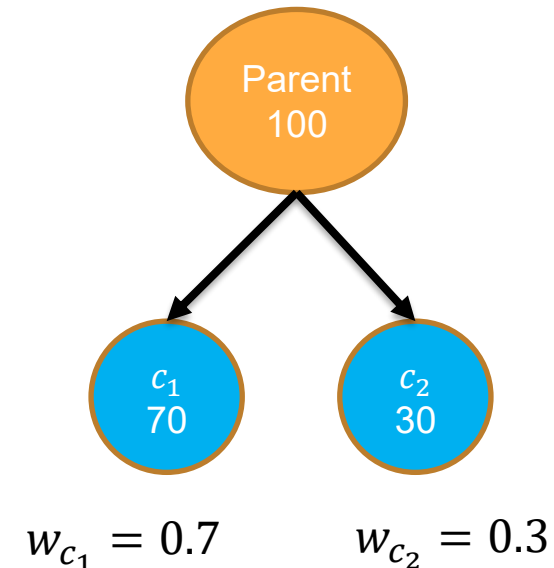
$Gini = 0$

Building a Decision Tree for Classification: Feature and Condition to Split

- **Good Split:** Total Entropy or gini index reduces after the split at a node
- **Information gain or Gini gain** is used to quantify this reduction in entropy or gini index
- **Information gain (or Gini gain):** Change in the entropy or gini index after the split (change in entropy from parent node to child nodes)

$$\Delta i = i(p) - (w_{c_1} * i(c_1) + w_{c_2} * i(c_2))$$

- w_{c_1} : Fraction of samples in child node c_1
- w_{c_2} : Fraction of samples in child node c_2
- **Objective:** To split the node such that the information or Gini gain is maximised



Building a Decision Tree for Classification: Feature and Condition to Split

- **Objective:** To split the node such that the information gain is maximised

$$\Delta i = i(p) - (w_{c1} * i(c_1) + w_{c2} * i(c_2))$$

- **Issue:** What if the number of possible splits is infinite – Features which take continuous values
- **Solutions:**
 1. Order the feature values and split at mid value of every pair
 2. Clustering the values in that feature into predefined number groups and taking mid values between cluster centres

$$x_1 = \begin{bmatrix} 1.2 \\ 1.5 \\ 2.4 \\ 2.7 \\ 5.6 \\ 5.9 \end{bmatrix}$$

Possible Splits:

$$x_1 \leq 1.35$$

$$x_1 \leq 2.55$$

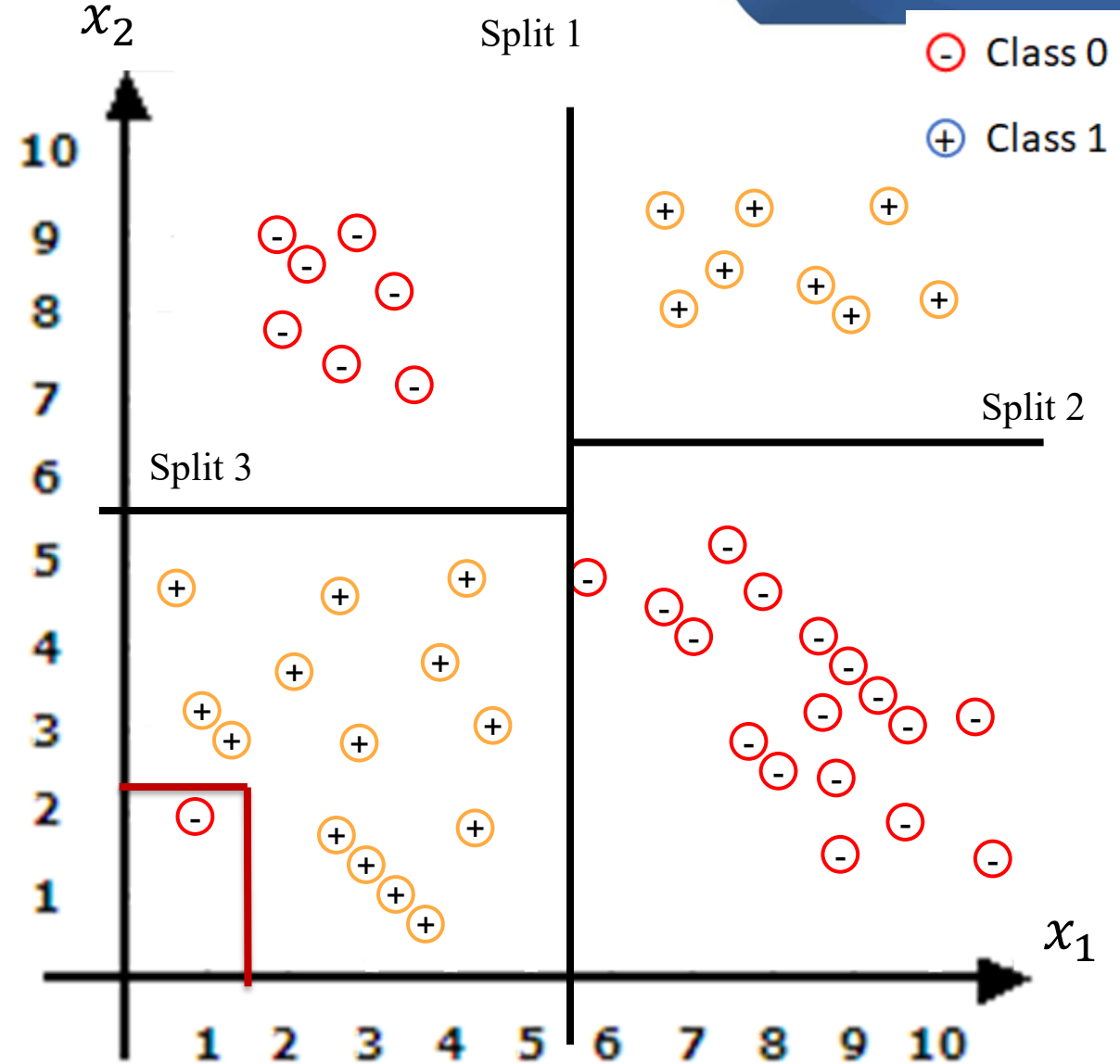
$$x_1 \leq 5.75$$

Building a Decision Tree for Classification

- Questions to be answered while building a decision tree:
 - How to select the feature and the condition to split upon at a given node?
 - When to stop splitting or fixing a node to be the leaf node ? What is the stopping criteria?

Building a Decision Tree for Classification: Stopping Criteria

- Ideally, splitting should stop when all the samples at a node belong to the same class i.e., the node is a leaf node
- **Issue:** Lack of generalisation when the leaf node contains very few samples – overfitting of data
- **Solutions:**
 - **Early stopping** – Stop building the tree after a predefined criteria is met
 - **Pruning** – After building a tree, remove the leaf nodes causing overfitting

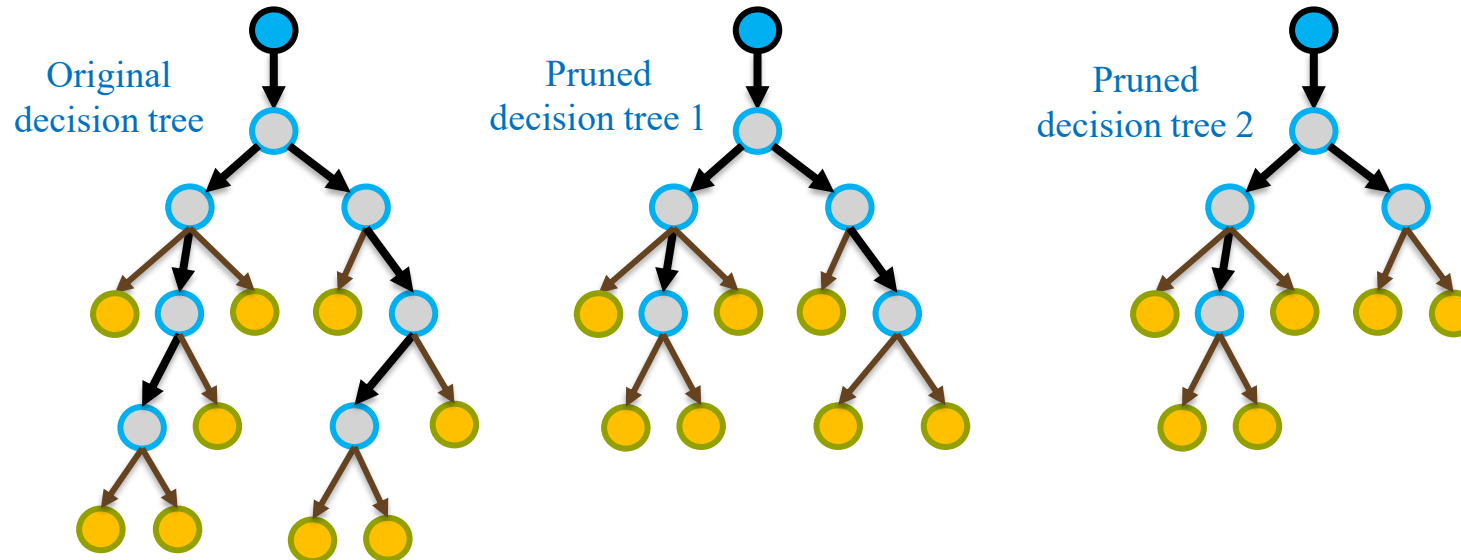


Building a Decision Tree for Classification: Stopping Criteria – Early Stopping

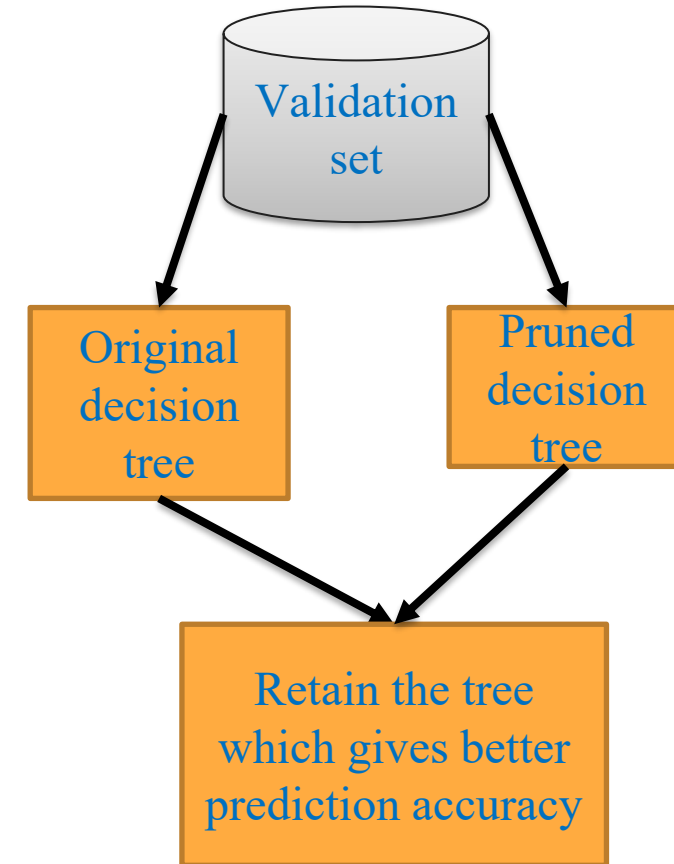
- Following are different early stopping techniques while building decision trees:
 - **Maximum tree depth:** Maximum number maximum layers for the tree can be pre-defined
 - **Minimum samples in a leaf node:** Pre-define the minimum number of samples that a leaf node may contain
 - **Minimum gain:** Pre-define the minimum information or Gini gain at a split below which no more splits are performed at the node
- In practice, all of these are combinedly used as stopping criteria

Building a Decision Tree for Classification: Stopping Criteria – Pruning

- **Pruning:** Removal of leaf nodes and branches on leaf nodes to reduce complexity of a tree
- **Validation set:** Used to validate if pruned tree performs better than original decision tree
- Build multiple trees with different nodes pruned and retain the one which performs best on validation set



Decision Trees and Variants



Building a Decision Tree for Classification: Summary of Steps

- **Input:** Training data with features values and target class labels
- **Step 1:** List the possible splits under each feature and calculate the information gain (entropy or gini index) when the split is performed
- **Step 2:** Split the tree based on the condition which results in maximum information gain – Feature space gets split into two subspaces
- **Step 3:** Perform further splits again based on maximum information or gini gain – Each of the feature subspaces get split further
- **Step 4:** Repeat until the stopping criteria is reached
- **Step 5:** Performing pruning using a validation set

Gini Index Vs Entropy – When to Use?

➤ Gini Index:

- Gini index is computationally simple and fast
- Preferred when computational efficiency is important (for large datasets)
- Used by default in many libraries
- Gini often favors splits that quickly increase node purity, especially by isolating a dominant class

➤ Entropy:

- Entropy is computationally expensive due to logs
- Can be slightly more sensitive to changes in class probabilities
- Sometimes prefers splits that reduce overall class uncertainty more strongly
- In practice, often gives splits similar to Gini

eg: Entropy changing more sharply

$$G(p) = 1 - p^2 - (1-p)^2 = 2p(1-p)$$

$$E(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

$$p=0.5 \quad G(p)=0.5 \quad E(p)=1$$

$$p=0.8 \quad G(p)=0.32 \quad E(p)=0.722$$

$$p=0.95 \quad G(p)=0.095 \quad E(p)=0.286$$

eg: 100 samples $\rightarrow$ Class A - 90 $P_A = 0.9$
 $\rightarrow$ Class B - 10 $P_B = 0.1$

Split: $C_1 = (0A, 9B)$
 $C_2 = (90A, 9B)$

$$G_1 = 0$$

$$G_2 = 1 - \left(\frac{90}{99}\right)^2 - \left(\frac{9}{99}\right)^2 = 0.1653$$

$$\text{Wgt Gain} = \frac{1}{100}(0) + \frac{99}{100}(0.1653) = 0.1636$$

Entropy: $H_1 = 0$
 $H_2 = 0.4395$

Weighted $H = \frac{1}{100}(0) + \frac{99}{100}(0.4395) = 0.4351$

Split 2: $C_1: (19A, 6B)$

$C_2: (71A, 4B)$

Gini: $G_1 = 1 - \left(\frac{19}{25}\right)^2 - \left(\frac{6}{25}\right)^2 = 0.3648$

$G_2 = 1 - \left(\frac{71}{75}\right)^2 - \left(\frac{4}{75}\right)^2 = 0.1008$

Entropy: $E_1 = 0.795$

$E_2 = 0.3004$

Wgt $E = \frac{25}{100}(0.795) + \frac{75}{100}(0.3004)$

Wgt $G_1 = 0.1636$

Wgt $G_2 = 0.1669$

$E^1 = 0.4351$

$E^2 = 0.424$

Decision Tree Building Algorithms

Classical Decision Tree Algorithms

- Different decision tree algorithms were developed to answer the same core questions in different ways:
 - How should the best split be selected?
 - Should the tree use binary splits or multiway splits?
 - Can the method handle continuous features?
 - Does it support classification only or regression also?
 - How should the tree be pruned to avoid overfitting?

Classical Decision Tree Algorithms

ID3 — Iterative Dichotomiser 3

Proposed by J. Ross Quinlan

One of the earliest and most influential tree-learning algorithms

Designed mainly for classification

Uses Information Gain for split selection

Allows multiway splits at categoricals

C4.5

Proposed by Ross Quinlan

Developed as an improvement over ID3

Still mainly for classification

Uses Gain Ratio, a normalized version of Information Gain; normalized by how fragmented is the split

Added important practical improvements such as handling continuous attributes, and pruning

Allows multiway splits at categoricals

CART — Classification and Regression Trees

Proposed by Breiman, Friedman, Olshen, and Stone

A broader framework than ID3/C4.5

Supports both classification and regression

Uses Gini index for classification and squared-error / variance reduction for regression

Binary splits only

Forms the basis of many modern tree implementations

Decision Tree Tuning

- **Goal of tuning:** Choose hyperparameters that control tree complexity and improve test performance
- **Common tuning parameters**
 - Maximum depth of the tree
 - Minimum samples to split a node
 - Minimum samples in a leaf
 - Minimum impurity decrease / minimum gain
 - Split criterion: Gini vs Entropy for classification

Advantages and Disadvantages of Decision Trees

Advantages:

- Easy to interpret and explain
- They do not require any domain knowledge for feature selection
- Can easily handle both numeric and categorical data
- Does not require any scaling
- The type of relation (linear or non-linear) between features and targets does not affect tree performance

Disadvantages:

- Decision tree learners can create complex trees that overfit the data and do not generalise well to unseen data
- Trees can be non-robust i.e., a small change in the data can cause a large change in the tree built

Regression Trees

Regression Trees

- Decisions trees are rarely used for regression tasks
- Differences of a regression tree from a classification tree:
 - I. **Measure of impurity:** Choose the split that minimises the sum of squared errors with respect to the mean target value in the child nodes
- Suppose the feature space is split into two regions (corresponding to two child nodes): r_1 and r_2

➤ Mean target value in r_1 : $\overline{y_{r_1}} = \sum_{i \in r_1} y_i / |r_1|$

➤ Mean target value in r_2 : $\overline{y_{r_2}} = \sum_{i \in r_2} y_i / |r_2|$

➤ Sum of squared errors wr.t. mean target:

$$e_{r_1} + e_{r_2} = \sum_{i \in r_1} (y_i - \overline{y_{r_1}})^2 + \sum_{i \in r_2} (y_i - \overline{y_{r_2}})^2$$

➤ Choose split which : $\min e_{r_1} + e_{r_2}$

Eg: Samples in region r_1 :

#	Target (y)
1	10
2	15
3	20
4	11

$$\begin{aligned}\overline{y_{r_1}} &= \frac{56}{4} = 14 \\ e_{r_1} &= (10 - 14)^2 + (15 - 14)^2 + (20 - 14)^2 + (11 - 14)^2 \\ e_{r_1} &= 62\end{aligned}$$

Regression Trees

- Regression trees are mostly used in popular ensemble methods
- Differences of a regression tree from a classification tree:
 1. **Measure of impurity:** Choose the split that minimises the sum of squared errors with respect to the mean target value in the child nodes
 2. **Predictions:** At a leaf node, the predicted value for a test sample is equal the mean of the target values of the samples present at that leaf node
 - Suppose r_l is the region corresponding to leaf node
 - Mean target value in r_l : $\overline{y_{r_l}} = \sum_{i \in r_l} y_i / |r_l|$
- Stopping criteria can be like the ones used in classification trees

Ensemble of Decision Trees

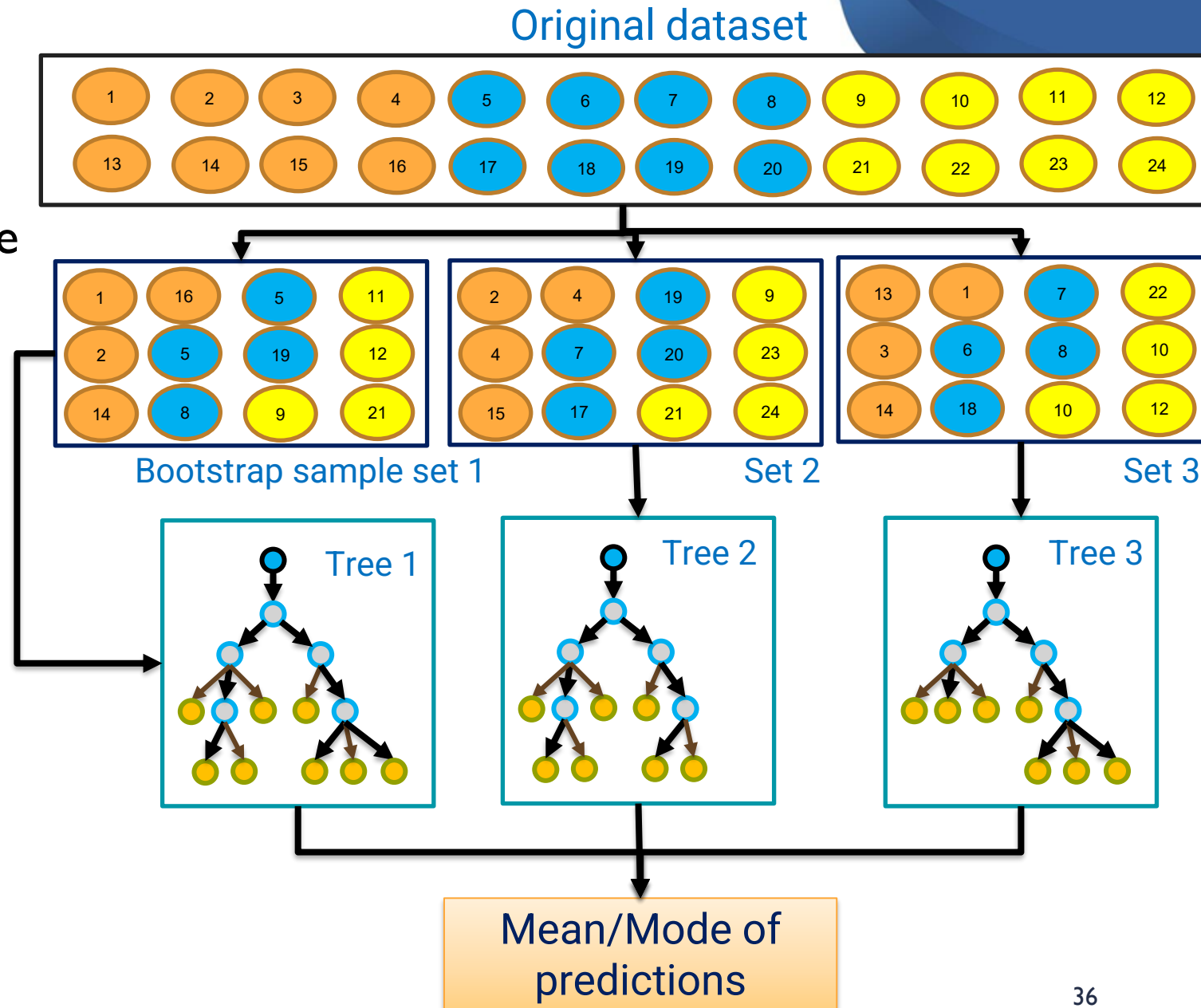
Why Ensemble Decision Trees?

- A single decision tree is:
 - highly sensitive to training data
 - unstable (small data changes can produce very different trees)
 - prone to overfitting, especially when deep
- **Idea:** Combine several decision trees to get better predictive performance – ensemble of decision trees
 - prediction variance can reduce
 - generalization improves
 - accuracy becomes much higher
- Ensemble Strategies:
 - **Parallel combination:** Build many independent trees to reduce variance – Bagging & Random Forest
 - **Sequential combination:** Build trees one after another with improvement to reduce bias - Boosting

Tree Bagging

- Steps in Tree Bagging:

1. Create multiple datasets from the given dataset by sampling data points with replacement - 'bootstrapping'
 2. Fit a decision tree to each re-sampled dataset
 3. On a test set, mean/mode of the predictions of all the trees is taken to make a prediction
- Bagging introduces randomness into samples of data



Why Does Bagging Improve Decision Trees?

- Bagging reduces prediction variance by averaging multiple trees

B trees :

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$$

- Variance of ensemble prediction is given by:

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

σ^2 : Variance of a tree prediction
 ρ : pairwise correlation of tree predictions

- Implication if trees are not perfectly correlated:

$$\rho < 1 \Rightarrow \text{Var}(\hat{f}_{\text{bag}}) < \sigma^2$$

- Bagging improves stability because:

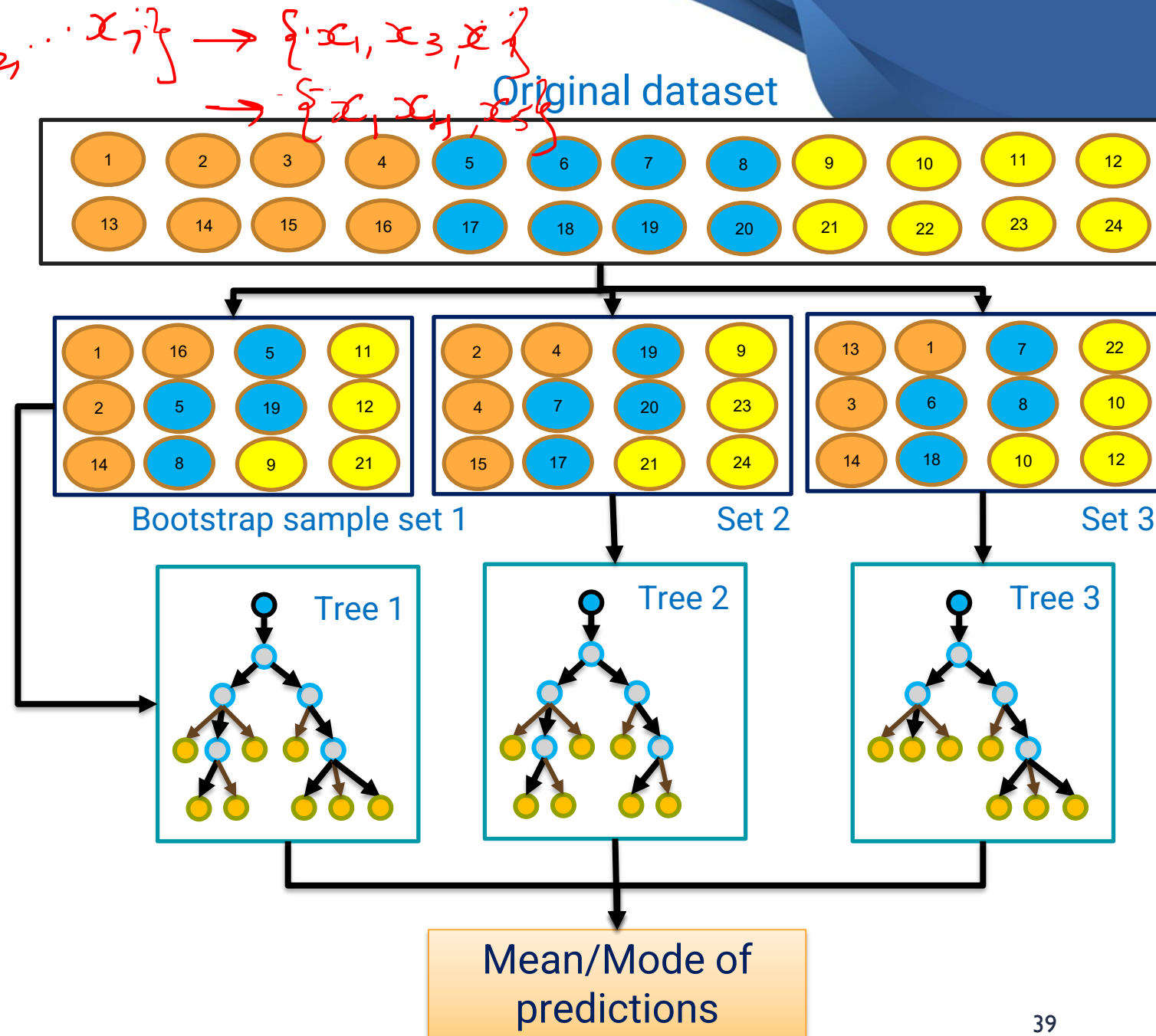
- decision trees are high-variance learners
- bootstrap samples create multiple slightly different trees
- averaging reduces fluctuations

Motivation for Random Forest

- Limitations of Bagging:
- Even after bootstrap sampling:
 - strong input features often dominate many trees
 - many trees become structurally similar
 - tree correlation (ρ) remains high
- Hence variance reduction is limited
- Q: How to reduce variance further?
- A: Introduce randomness in feature selection as well which reduces correlation between trees

Random Forest

- Ensemble of trees which are built in a similar manner to bagging
- However, at each node random subset of D features is sampled, and the best split is chosen only among those features
- Each split in Trees 1, 2 and 3 is based on gain observed at m features only
- Random forests introduce randomness into both samples and features
- Hence, the trees are more diverse and we get better predictions



Questions in Building Random Forest

- How many number of decision trees to create?
 - Sample datasets & build trees until the validation error no longer decreases
- How to choose the value of r (number of features)?
 - Typical defaults:
 - Classification $r = \sqrt{D}$
 - Regression $r = \frac{D}{3}$
- How deep to go?
 - Random forests usually go deeper than individual trees and pruning is not done as overfitting is handled by the ensemble

Intro to Tree Boosting

Motivation for Tree Boosting

- **Issue with Random Forest:**
 - Training of all trees happens in parallel
 - If many trees struggle with certain difficult patterns, there's no mechanism to correct their mistakes.
 - The final prediction is based on majority vote or averaging, but persistent errors across trees remain uncorrected.
- **Question:** Can we train models sequentially to focus on previous errors?
- **Idea of Boosting:** Learn from mistakes of earlier models
- Each model in the sequence tries to correct the errors made by the previous one i.e., each tree is dependent on previous ones

Boosting and Boosting Algorithms

- Simple Analogy for Boosting:
 - Suppose you are learning to play a song on a piano:
 - On your first attempt, you make mistakes.
 - The second time, you focus more on the notes you got wrong.
 - Each time you practice, you pay more attention to the parts you messed up earlier, and gradually the whole performance improves.
- Trees produced during boosting are called **boosted trees** and they reduce the bias of the model by correcting previous mistakes
- Different Boosting Algorithms:
 - Gradient Boosting
 - Extreme Gradient Boosting (XGBoost)
 - Light Gradient Boosting Machine (Light GBM)

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
= ("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
= bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exactly one mirror")
```

WILLIAM C. LEE

```
def mirror(modifier):  
    #add mirror to the selected  
    #object -mirror_x, mirror_y,  
    #mirror_z  
    mirror_ob = bpy.context.selected_objects[0]  
    mirror_mod = modifier
```

THANK YOU